UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
SEDE LATACUNGA

ARQUITECTURA DE SOFTWARE
NRC: 37761

Trabajo (Arquitectura Limpia)

Estudiante: Christopher Alexis Candelejo Fasso

Fecha: 23 de octubre de 2026

1. Introducción

El punto de partida es un sistema de facturación sencillo que funciona por consola. El objetivo de este trabajo no es solo agregarle funciones, sino comprobar algo más importante: que si el sistema está bien organizado, se pueden cambiar las piezas de afuera (cómo llegan las facturas o dónde se guardan) sin dañar las reglas del negocio.

Para comprobarlo se construyó un sistema de facturación en PHP con Arquitectura Limpia. El sistema recibe facturas de dos formas distintas (una API REST y un broker de mensajes), revisa reglas de fraude e impuestos, y guarda el resultado en una base de datos que se elige por configuración. La persistencia es real: usa SQLite, una base embebida que el enunciado permite en lugar de PostgreSQL u Oracle (que se representan con adaptadores simulados sobre SQLite). La API REST es un servidor HTTP funcional y el broker es una cola simulada, con un productor que publica y un consumidor que procesa después. Lo que importa es poder reemplazar cada pieza, no conectarse físicamente a PostgreSQL, Oracle o RabbitMQ.

2. La idea de la Arquitectura Limpia

En un sistema de facturación hay dos tipos de cosas. Por un lado está el **negocio**: qué es una factura, cuándo se rechaza y cuánto IVA lleva. Esto cambia poco. Por otro lado está la **tecnología**: la base de datos, la web, las colas de mensajes. Esto cambia seguido. Si ambas están mezcladas, cada cambio de tecnología obliga a tocar el negocio y se corre el riesgo de dañarlo.

La Arquitectura Limpia las separa en capas y pone una sola regla: **las capas de adentro no conocen a las de afuera**. Por ejemplo, la factura no sabe si se guardará en PostgreSQL u Oracle, ni si llegó por la web o por un mensaje. Quien se adapta es lo de afuera, nunca el negocio.

3. Cómo está organizado el sistema

El código se divide en tres capas. Las flechas de la figura muestran quién usa a quién: siempre apuntan hacia el centro.

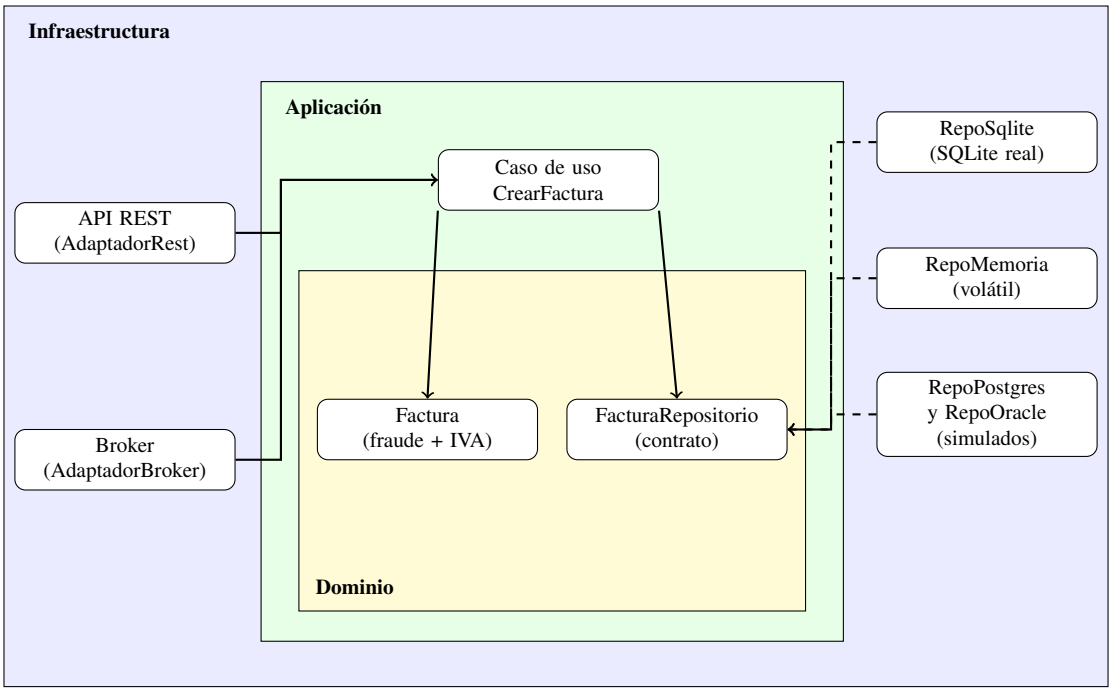


Figura 1: Diagrama de la Arquitectura Limpia del sistema. Las flechas continuas significan “usa a”; las punteadas significan “cumple el contrato de”.

Dominio (el centro). Tiene la factura con sus reglas de fraude e impuestos, los parámetros de esas reglas (`Reglas`) y el contrato `FacturaRepositorio`, que describe qué necesita el negocio para guardar y listar facturas. No depende de nada externo.

Aplicación. Tiene el caso de uso `CrearFactura`, que coordina los pasos: crear la factura y guardarla (y `ListarFacturas`, para consultarlas). No sabe quién lo llama ni dónde se guarda el resultado.

Infraestructura (lo de afuera). Tiene las piezas que se pueden cambiar: las dos formas de recibir facturas (REST y broker), las formas de guardarlas (SQLite, memoria y los adaptadores de PostgreSQL y Oracle) y la lectura de la configuración (`main.php` y la carpeta `config`).

4. Cómo se resolvió cada punto del enunciado

4.1. Recibir facturas por dos vías distintas

La API REST y el broker son diferentes por fuera, pero hacen lo mismo por dentro: reciben un texto en formato JSON, lo convierten en una lista de datos y llaman al mismo método, `CrearFactura::ejecutar`. No tienen reglas de negocio. La diferencia es el momento: la API REST (`api.php`) responde en la misma petición, mientras que el productor deja el mensaje en la cola y termina, y el consumidor lo procesa después. Gracias a esto la lógica de crear facturas existe una sola vez, y el caso de uso no sabe si los datos llegaron por la web o por un mensaje. Ese método es el punto de entrada a la capa de aplicación.

4.2. Proteger las reglas del negocio

La lógica de las reglas vive solo en la clase `Factura`. Sus valores (límite, lista de bloqueados, IVA) están en el archivo `config/reglas.json`, así que se ajustan sin modificar código. Son reglas simuladas, pero cumplen su papel:

- **Control de fraude:** se rechaza una factura si el monto supera el límite configurado (10 000), si el monto no es positivo o si el cliente está en la lista de bloqueados.
- **Política fiscal:** el IVA configurado es del 15 %, y de 0 % para clientes exentos.

Estas reglas se revisan al crear la factura. Si no se cumplen, la factura simplemente no se crea. Por eso el negocio queda protegido sin importar quién lo use: ni la base de datos ni la web pueden saltárselas, porque no las conocen.

4.3. Guardar en cualquier base de datos

El dominio define un contrato, `FacturaRepositorio`, con dos operaciones: guardar y listar facturas. `RepoSqlite` lo cumple con una base SQLite real; `RepoMemoria`, con un arreglo volátil; y `RepoPostgres` y `RepoOracle` son adaptadores simulados que, como permite el enunciado, usan SQLite por debajo. El valor `persistencia` del archivo `config/config.json` decide cuál usar. Cada una numera sus facturas de forma distinta (`FAC-1`, `MEM-1`, `PG-1`, `ORA-1`), lo que permite ver cuál se está usando. El contrato pertenece al dominio, así que es la base de datos la que se adapta al negocio y no al revés.

5. Cómo se demuestra que se puede cambiar sin tocar el núcleo

El objetivo final del trabajo es poder cambiar el broker, la base de datos o la interfaz web sin modificar una sola línea de las capas de dominio y aplicación. Para comprobarlo se calcula la huella (MD5) de los archivos de esas dos capas antes y después de cada cambio. Si la huella es igual, el archivo no cambió ni un carácter.

Cambio	Qué se modificó	Dominio y aplicación
Base de datos	Un valor en `config/config.json`: de `"sqlite"` a `"oracle"`. La factura pasó de `FAC-1` a `ORA-1` con el mismo total (1150).	Huellas idénticas.
Broker	Se agrega un adaptador nuevo (por ejemplo para Kafka) en infraestructura, que llama a `CrearFactura::ejecutar` igual que `AdaptadorBroker`.	Huellas idénticas.
Interfaz web	Se hace igual: un adaptador nuevo en infraestructura, como el de REST.	No hay nada que cambiar en ellas.

Cuadro 1: Resultado de cada cambio.

Además, el archivo `prueba.php` ejecuta 15 verificaciones y todas resultan correctas: las reglas de IVA y fraude funcionan sin una base de datos real; cambiar los valores de `reglas.json` cambia el resultado sin tocar código; la misma factura enviada por REST y por broker da el mismo total y queda en la misma base; los datos persisten en SQLite; el resultado de negocio es el mismo con las cuatro persistencias; y los archivos de dominio y aplicación no mencionan PDO, SQL, JSON, HTTP ni colas.

6. Ventajas reales de esta arquitectura

- **Cambiar de tecnología es barato.** Cambiar de base de datos o de broker se reduce a escribir una pieza nueva o cambiar un valor, sin tocar el negocio.
- **Las reglas están en un solo lugar.** Si cambia el IVA o el límite de fraude, se edita `reglas.json` y todas las entradas lo reflejan.
- **Se prueba más fácil.** El negocio se puede probar sin base de datos, sin servidor web y sin broker.
- **No se repite código.** Agregar una nueva forma de recibir facturas no obliga a copiar la lógica de facturación.
- **Menos riesgo.** Un problema en la tecnología externa no puede dañar las reglas del negocio.

7. Conclusión

La separación en capas permite cambiar la base de datos, el broker o la interfaz web sin modificar el dominio ni la aplicación, y esto se comprobó con las huellas de los archivos. El precio es un poco más de estructura al comenzar (un contrato y varios adaptadores), pero ese esfuerzo se recupera cada vez que el sistema necesita cambiar.